

สรุปเนื้อหาในคาบวันที่ 7 กรกฎาคม พ.ศ. 2569 ฟังก์ชัน หน่วยความจำ และการแก้ปัญหา แนวคิด Divide and Conquer เป็นหัวใจของการแก้ไขปัญหาวงวิศวกรรมคอมพิวเตอร์ เนื่องจากเป็นวิธีที่แก้ไขปัญหาค่อยๆ ง่ายโดยการแบ่งปัญหาย่อยๆ และแก้ไขทีละปัญหา ตัวอย่างเช่น หากต้องการสร้างรถยนต์ทั้งคัน มันดูยากเกินไป แต่ถ้าแบ่งออกเป็น การทำล้อ ทำเครื่องยนต์ และทำระบบไฟ คุณสามารถเริ่มแก้ไขปัญห จากส่วนที่ง่าย ที่สุดก่อน เช่น ทำส่วน B ให้เสร็จเพื่อสร้างกำลังใจ แล้วค่อยทำส่วนที่เหลือจนครบ การประยุกต์ใช้ในมุมมองของการเขียน program เราจะนำปัญหาหรือความต้องการของ requirement user นำมาเขียนโปรแกรมโดยเริ่มจากการเขียน work flow / flow chart แล้ว จึงนำมาเขียนเป็น function ย่อยๆ ที่ยกตัวอย่างได้ดีคือการทำ recursive function ซึ่งจะ เป็นฟังก์ชันที่เรียกตัวมันเองแต่ละครั้งทีเรียกตัวมันเองจะค่อยๆ ทำให้ปัญหาเล็กลง หรือการ ย่อยปัญหา เช่น ตัวอย่างง่ายๆ เรื่อง factorial คือการคูณทางคณิตศาสตร์เช่น 5! คือ $5 \times 4 \times 3 \times 2 \times 1$ เราจะได้ 120 หากให้อธิบายง่ายๆ มันคือกล่องในกล่องโดยให้เรามองกล่องเป็น ฟังก์ชันจะเป็นวิธีมาตรฐานเริ่มต้นเพื่อเรียนรู้ factorial ซึ่งสำหรับผมมองเป็นในลักษณะของ การโยงเส้นแล้ว tab indent ย่อหน้าแบบที่ครู toto ทำให้ผมดู

Reusability เขียนครั้งเดียวแต่จะเรียกใช้งานกี่ครั้งก็ได้, Maintainability ง่ายต่อการดูแล หาก เกิดข้อผิดพลาดเราสามารถเข้าไปดูในฟังก์ชันนั้นได้ ง่ายกว่ามานั่งมโนโค้ดเอาเอง ตัวอย่างใน ชีวิตจริง คือเช่น รถเสียถ้ารู้ว่าเป็นที่ battery เราก็เปลี่ยนแค่ batter อย่างเดียวนั่นก็พอ Abstraction การซ่อนความซับซ้อน เราสามารถเรียกใช้งานฟังก์ชันโดยที่เราไม่จำเป็นต้องรู้ ว่า การทำงานเบื้องหลังจริงๆ เป็นอย่างไร, Readability ทำให้โค้ดสะอาดและเข้าใจง่าย เห็น โครงสร้างสร้างโปรแกรมเป็นลำดับขั้นชัดเจน

ความแตกต่างระหว่างภาษา C และ Python เป็นภาษาที่มีระเบียบและเป็นระนาบเดียวกัน มีความซับซ้อนในเรื่องของ Data Type เช่น int, float, char การประกาศตัวแปรที่ชัดเจนกว่า แม้กระทั่งเรื่องของการจัดพื้นที่ ถูกออกแบบมาสำหรับคนขี้เกียจ แต่ภาษา C นั้นมี

ความเร็วและมีประสิทธิภาพมากกว่าต่างกับ python ที่ทำงานได้ช้ากว่า ภาษา C จะใช้ compiler ส่วนภาษา python จะใช้ interpreter

การจัดการหน่วยความจำและด้วย pointer และ % ในมุมมองของคอมพิวเตอร์จะมี buffer memory อยู่ซึ่งแต่ละ buffer ก็จะมี memory ซึ่งทุกตัวแปรที่เราประกาศจะสามารถเข้าถึงมันได้เช่น หากเรากำหนดให้ตัวแปร a เป็น 10 แต่เราส่งค่า address ให้ x เราสามารถบอกให้ x นั้นที่รู้ตำแหน่งเอาเลขใหม่ไปแทนเลข 10 แทน หากยกตัวอย่างให้เห็นภาพง่ายๆ คือ สมมุติเราต้องการเปลี่ยนแอร์ที่บ้านเราก็ต้องบอกช่าง(เปรียบเหมือนตัวแปรที่เก็บ address) ว่าแอร์ที่เราต้องการให้เปลี่ยนอยู่ที่ไหนเช่น ชั้น 2 ก็เปลี่ยนแอร์ที่ชั้น 2 เมื่อเราซ่อมเสร็จแอร์ก็จะเปลี่ยนสภาพไปตามที่ช่างได้ทำเอาไว โดยเราสามารถนำไปประยุกต์ใช้กับ function ได้ โดยที่จะไม่ return ค่าเลยก็ได้เพียงแต่จะต้องรู้ address ก็สามารถเปลี่ยนแปลงได้เลย

Recursive function การเรียกซ้ำอันทรงพลังไม่พึ่ง loop

คือกระบวนการเรียนรู้ใช้ตัวเองเพื่อแก้ไขปัญหา โดยในแต่ละครั้งของการเรียกใช้ตัวเองจะค่อยๆ ลดปัญหา ลงไปเรื่อยๆ ตัวอย่างเช่นเรื่องของตุ๊กตารัสเซีย matryoshka หากคุณต้องการหาว่ามีตุ๊กตาก็ตัว คุณต้องเปิดตัวใหญ่เพื่อเจอตัวที่เล็กลงไปเรื่อยๆ จนถึงตัวสุดท้ายที่เปิดไม่ได้แล้ว โอเคพูดมาถึงตรงนี้แล้วอีกเรื่องเลยที่ผมลืมพูดถึงไป เรื่องของการเรียก prototype คือการให้โปรแกรมเนียร์รู้จัก function ก่อนที่จะเรียกใช้งานมัน อาจจะดูงงหน่อยแต่อย่าลืมโปรแกรมนั้นอ่านจากบนลงล่าง นอกจากจะให้มันรู้จักแล้วเราจะต้องสร้างฟังก์ชันนี้มันขึ้นมาก่อนด้วยนะ ไม่นั้นมันก็ไม่จะร้องไห้อายว่าฟังก์ชันนี้ไม่เห็นมีอะไรเลยนายจะให้เราทำอะไรละ โอ้ลืมไปอีกเรื่องสำคัญมากๆ เลยคือ global variable และ local variable ถ้าให้พูดถึง global variable คือตัวแปรที่ทุกฟังก์ชันสามารถเข้าถึงได้ส่วน local variable คือจะถูกประกาศภายในฟังก์ชันนั้นและฟังก์ชันนั้นจะเข้าถึงได้ ยกเว้นมีการส่งค่าและ return คืนกลับมาแต่ไม่ใช่มันส่งให้กับ function นั้นโดยตรงนั้นแต่เป็นการที่เราประกาศตัวแปรในอีกฟังก์ชันก่อนแล้วค่อย copy ค่าลงไป parameter ของอีก function จากนั้นภายในฟังก์ชันนั้นก็

จะต้องเอาตัวแปรพวกนี้ไปทำใน statement block ที่เขียนแล้วสุดท้ายก็จะ return ค่าเป็น value /variable ออกมาก็ได้ นอกจากเรื่องของการส่งค่าและ return ค่าแล้วเราจะต้องเข้าใจว่าการส่งค่าเข้าไปเราเรียกข้อมูลที่ส่งเข้าไปว่า argument และเราเรียกข้อมูลที่รับว่า parameter

```
#include <stdio.h>

void hello(int n, char *name){
    for (int i = 0; i < n; i++){
        printf("%2d) Hello, %s.\n", i + 1, name);
    }
}

void hello1(int n, char *name){
    int i = 0;
    while(i < n){
        printf("%2d) Hello, %s.\n", i + 1, name);
        i++;
    }
}

int hello2(int n, char *name, int i){
    if (n <= 0){
        return i;
    }else{
        printf("%2d) Hello, %s.\n", i + 1, name);
        i++;
        n = i + hello2(n - 1, name, i);
        return n;
    }
}

int main(void){
    int n;
    char *name = "Kunloto";
    //char * name = "Piyawat"

    printf("Enter a positive interger");
    scanf("%d", &n);
    printf("hello\n");
    hello(n, name);
    printf("hello1\n");
    hello1(n, name);
    printf("hello2\n");
    hello2(n, name, 0);

    return 0;
}
```